Design and Evaluation of an Accessible IDE Plugin for Supporting Neuro-divergent Software Developers

Clo Elis

220402088

May 2026

Computer Science

Supervisor: Dr. Vlad Gonzalez

Abstract (NEED TO BE REWRITTEN) Modern Integrated Development Environments (IDEs) are designed for maximum functionality, yet their high feature density creates significant barriers for neurodivergent software developers. Individuals with Attention-Deficit/Hyperactivity Disorder (ADHD), Autism Spectrum Disorder (ASD), or dyslexia often face challenges related to executive dysfunction, including cognitive overload, difficulty initiating tasks, and sensitivity to interruptions in workflow. This research explores the extent to which an Artificial Intelligence (AI) integrated IDE plugin can mitigate these challenges.

The project involves the development of a custom VS Code extension featuring three core modules: a Minimalist Toggle to reduce interface-driven distractions, a Task Architect that uses Large Language Models (LLMs) to decompose complex prompts into manageable checklists, and a Visual Styler providing high-contrast, low-strain presets. The effectiveness of the plugin is evaluated through a comparative study using a baseline VS Code "Vanilla" versus the enhanced environment. Success is measured via the NASA Task Load Index (NASA-TLX) [3] to quantify mental demand and the System Usability Scale (SUS) [7] to assess overall tool effectiveness.

By streamlining the development environment and providing AI-assisted task structuring, this project aims to lower the "barrier to entry" for neurodivergent developers, allowing them to reallocate cognitive resources from navigating complex tools to solving core programming problems.

Declaration “I declare that this document represents my own work except where otherwise stated” - Clo Ellis

Acknowledgments

Contents

List of Abbreviations **V**

Glossary **VI**

1 Introduction **1**

1.1 Motivation 1
1.2 Problem Statement 1
1.3 Understanding Neurodiversity in Software Development 2
1.4 Aims and Objectives 3

2 Literature Review **5**

2.1 Cognitive Load Theory (CLT) 5
2.2 Neurodiversity in Software Engineering 7
2.3 The “Double-Edged Sword” of Hyperfocus 8
2.4 Existing Tools 9
2.5 Colour filters 10
2.6 AI as a Cognitive Tool: Task Decomposition and Scaffolding 11
2.7 NASA-TLX and SUS: Measuring Cognitive Load and Usability 11

3 Design and Planning **13**

3.1 User Requirements 13
3.2 Module Design 13
3.3 System Architecture 15
3.4 Implementation 17
3.5 Technology Stack 17
3.6 UI Simplification and Minimalist Toggle 17
3.7 AI Integration 17
3.8 Inital User Testing and Feedback 17
3.9 Evaluation and Methodology 19
3.10 Experimental Design 19
3.11 Quantitative and Qualitative Frameworks 20
3.11.1 NASA Task Load Index (NASA-TLX) 20
3.11.2 System Usability Scale (SUS) 20
3.12 Data Collection and Analysis 21

4 Results and Discussion **22**

4.1 Analysis of NASA-TLX Scores 22
4.2 Task Initiation Findings 22
4.3 Discussion 22
4.4 Limitations 22

References **23**

List of Abbreviations

ADHD Attention-Deficit/Hyperactivity Disorder. 1–3, 7–10, I, V

AI Artificial Intelligence. 1, 3, I, V

API Application Programming Interface. 3, 14, V

ASD Autism Spectrum Disorder. 1, 2, 7, I, V

CoT Chain of Thought. 3, 10, 12, 15, V

EL Extraneous Load. 5, 6, V

GL Germane Load. 5, 6, V

HITL Human-in-the-Loop. 11, V

IDE Integrated Development Environment. 1–3, 6, 7, 9, 11, 16, I, V

IL Intrinsic Load. 5, 6, V

LLM Large Language Model. 3, 10, I, V

NASA-TLX NASA Task Load Index. 4, I, V

SUS System Usability Scale. 4, I, V

UI User Interface. 3, V

WCAG Web Content Accessibility Guidelines. V

Glossary

. 2, V

executive dysfunction A range of cognitive difficulties affecting the ability to plan, focus attention, and multi-task. 1–3, 7, 8, I, V

hyperfocus A state of intense focus and concentration, often seen in individuals with ADHD, where they become deeply engrossed in a task or activity, sometimes to the exclusion of everything else. 1, 3, V

neurodivergent A term used to describe people whose brains function, learn, and process information differently than what is considered typical. 1–3, I, V

task paralysis A condition where an individual becomes unable to initiate or complete a task, often due to cognitive overload or anxiety. 2, V

1 Introduction

1.1 Motivation

This project was initially motivated by my lived experience as a neurodivergent developer and by personal challenges in using industrial software development tools. In professional settings, my ASD was often framed as a "superpower" due to my ability to rapidly acquire new technical skills and engage in deep logical analysis. However, this perceived strength came with a significant cognitive tax. I found adapting to new environments and maintaining a sustainable work-life balance extremely difficult, particularly when faced with task initiation and the decomposition of complex programming problems. Upon researching existing methodologies and consulting with other neurodivergent developers, it became clear that current Integrated Development Environments (IDEs) offer no effective, "out-of-the-box" solutions for managing these executive functioning barriers. This gap in assistive technology motivated the development of a solution that transitions the IDE from a passive tool to an active cognitive partner.

Existing IDEs can lead to increased cognitive overload and difficulty in maintaining focus. By streamlining interfaces and providing AI-driven task decomposition, this project aims to reduce the strain on neurodivergent individuals in both professional and educational settings, as developers often report challenges with cognitive overload, hyperfocus, and breaking down more complex problems when programming

1.2 Problem Statement

IDEs are a primary coding tool for modern software development, yet there are still elements of the interface that remain a significant barrier for neurodivergent developers, specifically those with ADHD, ASD, and dyslexia. While these tools are feature-rich, they are designed with a "neurotypical bias" that assumes a high baseline for filtering distractions and managing multi-step planning.

Current IDEs prioritize high feature density, often presenting a constant stream of visual stimuli, including nested menus, status bars, and dense technical jargon. For developers who experience sensory overwhelm, this environment creates an immense Extraneous Cognitive Load [15]. This "visual noise" competes for limited working memory, often exhausting a developer's "cognitive budget" before they can engage with the actual logic of the code.

Research indicates that developers with ADHD frequently struggle with task paralysis during the initiation phase of a project [2]. Furthermore, those with ASD may face challenges with high-level strategic planning and the decomposition of abstract requirements

into actionable steps. Constant notifications—such as linter warnings or compiler recommendations—can inadvertently trigger hyperfocus on peripheral details, leading to "rabbit holes" that derail the core development goal.

While AI tools are increasingly used in software development, they often focus on generating code rather than supporting the process of being a developer. There is a lack of "out-of-the-box" inclusivity in these tools, forcing neurodivergent individuals to manually create complex custom configurations to remain productive. Consequently, there is a critical need for an IDE intervention that acts as a "digital ramp", one that filters environmental noise and provides the necessary scaffolding to bypass the freeze response common in task paralysis.

1.3 Understanding Neurodiversity in Software Development

Before going into the specific technical barriers of modern IDEs, it is necessary to define neurodiversity and the cognitive frameworks that underpin this research. Originally coined by Judy Singer in the late 1990s, "neurodiversity" views neurological differences—such as ASD, ADHD, and dyslexia, as natural variations in the human genome rather than purely medical deficits [14].

In the context of software engineering, these variations often manifest as unique cognitive processing styles. While many neurodivergent developers possess high aptitude for logic and pattern recognition, they frequently encounter barriers related to executive dysfunction. This is an umbrella term for difficulty in mental processes that enable us to plan, focus attention, remember instructions, and juggle multiple tasks successfully. For a developer, executive dysfunction is not a lack of technical ability, but rather a barrier in three key areas: Inhibitory Control (filtering IDE "noise"), Working Memory (tracking complex syntax), and Cognitive Flexibility (switching between high-level logic and low-level code).

This challenge is further complicated by the "Double Empathy" problem—the idea that communication breakdowns occur because different groups process information differently, rather than one group being inherently "wrong". In the context of this project, neurodivergent developers may struggle with "implied" logic in a team setting, where their interpretation of a task differs from the team's original plan. However, if instructions are provided clearly and literally, these challenges can be mitigated. Furthermore, while neurodivergent developers often utilize hyperfocus to solve complex problems, this state is inherently fragile. A single interruption—such as a non-essential notification—can lead to a total loss of mental context, carrying a "state recovery cost" that significantly hinders productivity. By identifying these specific cognitive roadblocks, we can move away from a medical model of disability and toward a social model that adapts the development environment to the user's brain.

To provide a clearer lens for the objectives of this project, the below table outlines the specific strengths and environmental challenges associated with the conditions most prevalent in the developer community.

Table 1: Neurodivergent Strengths and Environmental Challenges

Condition	**Potential Strengths**	**Common Environmental Challenges**
ASD	Deep focus, systematic thinking, high integrity, exceptional memory for details, strong pattern recognition.	Sensory overload in noisy or bright environments, misinterpretation of direct communication style, difficulty with unspoken social rules.
ADHD	Creativity, hyperfocus on passions, adaptability, high energy, ability to multitask or rapidly switch contexts.	Distractibility in chaotic settings, struggles with time management and organisation, impulsivity in structured environments.
Dyslexia	Strong visual-spatial reasoning, excellent big-picture thinking, creative problem-solving, strong verbal skills.	Difficulties with text-heavy tasks, timed reading assignments, and conventional written communication.

1.4 Aims and Objectives

Project Aim

The primary objective of this project is to design, implement, and evaluate an assistive VS Code plug-in that reduces extraneous cognitive load and mitigates the effects of executive dysfunction for neurodivergent software developers. By introducing a "cognitive prosthetic" layer, the project seeks to determine if targeted User Interface (UI) simplification and AI-driven task scaffolding can measurably improve the developer experience.

Research Objectives

Objective 1: Interface Optimisation (The Minimalist Toggle)

Target: Develop a single "Minimalist Toggle" feature that simultaneously hides non-essential UI elements, including the Activity Bar, Status Bar, Mini-map, and Breadcrumbs.

Justification: This directly addresses the issue of visual overload by reducing the number of stimuli competing for the developer's attention, thus lowering the Extraneous Cognitive Load and creating a more focused coding environment [15], reducing visual noise that leads to distraction and sensory overwhelm for autistic and ADHD developers.

Objective 2: AI-Driven Task Decomposition (The Task Architect)

Target: To implement an LLM-integrated module that translates ambiguous programming goals into structured, editable checklists using a Chain of Thought (CoT) prompting strategy. This should be implemented via the OpenAI Application Programming Interface (API) that breaks down high-level requirements without providing the direct code solution [11].

Justification: This supports "Task Initiation" and reduces the energy required to overcome task paralysis, a primary barrier for developers with ADHD and ASD [4].

Objective 3: Simplification of Technical Feedback (The Code Explainer)

Target: To build a diagnostic interceptor that re-interprets complex compiler errors into plain-language, bulleted instructions [12]. Limit output to a maximum of three bullet points using a "Direct Instruction" style.

Justification: This minimizes the cognitive demand of interpreting dense jargon, preventing "rabbit holes" and supporting the literal communication needs identified in the Double Empathy problem. By simplifying the feedback loop, developers can maintain focus on the core problem rather than getting lost in peripheral details. This is particularly beneficial for those with ASD, who may struggle with implied logic and prefer clear, direct instructions. Additionally, it can help mitigate the hyperfocus trap by providing concise guidance that keeps the developer on track without overwhelming them with information.

Objective 4: Visual Accessibility Customisation (The Visual Styler)

Target: To create at least two visual presets, such as "Soft High Contrast", that adhere to WCAG 2.1 guidelines to reduce eye strain and enhance readability [1].

Justification: Addresses the sensory sensitivities and "attractive colour" preferences reported by neurodivergent developers in research surveys [12].

Objective 5: Empirical Evaluation of Cognitive Impact

Target: To conduct a comparative study using the NASA-TLX and the SUS to validate the plugin's efficacy against a standard IDE.

Justification: This provides a standardized scientific metric to confirm if the tool successfully reduces mental demand and frustration for the target demographic.

2 Literature Review

2.1 Cognitive Load Theory (CLT)

Cognitive Load Theory, primarily developed by John Sweller, suggests that since the human brain has limited working memory capacity, instructional design can aid in avoiding overload.

Sweller's Tripartite Framework of Load

Sweller [15] proposes a three-part load framework that identifies three different types of cognitive load that impact a learner's ability to process information:

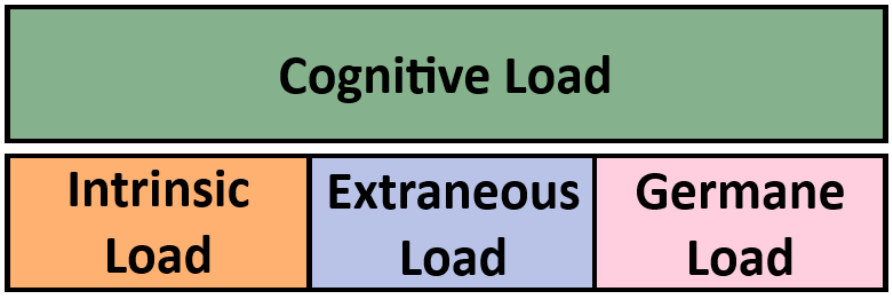

Figure 1: Sweller's Tripartite Model of Cognitive Load, illustrating the balance between Intrinsic, Extraneous, and Germane loads.

The framework can be simplified down to the idea that the Total Cognitive Load (TCL) is the sum of Intrinsic Load, Extraneous Load, and Germane Load. The goal of instructional design is to manage these loads effectively to maximise learning outcomes.

$$TCL = IL + EL + GL$$

where the goal of your plugin is to satisfy the condition:

$$\downarrow EL + \text{scaffold } IL = \uparrow GL$$

1. Intrinsic Load (IL): This is the inherent difficulty of the task itself. It is defined by the number of elements must be processed simultaneously to understand a concept. For example, learning basic vocabulary has low intrinsic load, while mastering complex syntax has high intrinsic load.

2. Extraneous Load (EL): This is seen as the unnecessary load that you would aim to minimise wherever possible. It is influenced by the way information is presented. Some examples that could exacerbate the load are poorly designed diagrams, dis-

tracting environments or redundant text. Poor design choices force the brain to waste resources on non-essential processing.

3. Germane Load (GL): This is the load I am aiming to maximise. It refers to the mental effort that can be directed toward the more key task, which in this scenario would be the code programming task.

One of the main focuses of this project is that for neurodivergent developers, the IDE environment often imposes a disproportionate Extraneous Load. When EL is high, the "cognitive budget" is exhausted before the developer can engage with the core logic of the code, leading to task paralysis. My plugin aims to "free up" this mental bandwidth by suppressing EL (via the Minimalist Toggle) and scaffolding IL (via the Task Architect), thereby maximizing the availability of cognitive resources for Germane processing. By reducing the noise and providing structured support, the plugin allows neurodivergent developers to focus their cognitive resources on the actual problem-solving aspect of programming, rather than being overwhelmed by the interface or the complexity of the task at hand.

Integration with Rosenshin's Principles of Instruction

While Sweller provided the theoretical framework, Barack Rosenshine provided a "how-to" to be used in classrooms or digital environments [13].

Key intersections include:

- Small Steps (Principle 2): By presenting new material in small increments, the instructor limits the Intrinsic Load, ensuring the working memory is not overwhelmed at the outset.
- Scaffolding (Principle 7): Providing temporary supports reduces Extraneous Load, allowing the learner to focus their cognitive resources on the core task rather than the "noise" of the process.

To view these from more of a development lens, principle 2 supports breaking down the core task into smaller, more manageable steps. Further more these smaller task can be seen as "scaffolding" for the user when they are beginning a task without handing them the answer directly.

The Cognitive Theory of Multimedia Learning (CTML)

While Sweller focuses on the load, Richard Mayer's CTML provides a more detailed look at how visual and auditory channels process information. In the context of a software developer, the IDE is a high-density multimedia environment.

The Split-Attention and Redundancy Principles by Mayer (2021) identifies the Split-Attention Effect, which occurs when learners must divide their attention between multiple

sources of information that are physically separated but cognitively related. For a neurodivergent developer, an error in the console that refers to a line of code 500 lines above creates a massive “spatial split”. The mental effort required to integrate these two sources is entirely Extraneous Load.

2.2 Neurodiversity in Software Engineering

To best design an inclusive plug-in, I needed to gain a deep understanding of the specific cognitive roadblocks faced by neurodivergent developers. Recent research by Verma et al. [16] and Gama et al. [4] highlights that developers with ADHD, ASD, and Dyslexia face significantly higher frequency of interruptions and difficulty obtaining knowledge.

The Cognitive Load Connection

For a neurodivergent developer, a standard IDE error message often carries a high Extraneous Load. What a neurotypical developer might perceive as a simple notification, a neurodivergent individual may experience as a wall of cryptic text, inconsistent highlighting, and ambiguous phrasing. This noise competes for limited working memory resources, leaving little room for the Intrinsic Load required actually to solve the logical problem.

Executive Functioning and Task Switching

executive dysfunction is a primary barrier cited by both [16] and [4]. This manifests itself as difficulty in “task initiation” (starting a new project) and “set-shifting” (switching between tasks).

The Industry Context: In a modern software environment, developers are rarely afforded the luxury of a single focus; they must balance code reviews, meetings, and multiple projects.

The Impact: For those with ADHD, the mental cost of “re-loading” the context of a project after an interruption is significantly higher. If the IDE does not provide clear "waypoints" or historical context, the developer may become cognitively paralysed, unable to determine where they left off.

Specific Challenges by Condition

Based on the synthesis of recent studies, the challenges can be categorised as follows:

ADHD: Frequent loss of“state” due to external interruptions and internal distraction. [16] found that these developers report higher stress related to “waiting for answers,” as delays in information retrieval often lead to a total loss of task momentum.

ASD: Challenges with the “Double Empathy” problem in code—where the high-level abstractions or “implied logic” used by neurotypical colleagues create a high cognitive overhead for autistic developers who prefer explicit, literal documentation.

Dyslexia: Significant hurdles during the "syntax-heavy" stages of programming. While these developers often excel at high-level architectural mapping, the Extraneous Load of spelling-sensitive compiler errors and dense documentation can be a major deterrent to productivity.

Beyond Medical

To understand why existing tools fail, we must examine the shift from the Medical Model to the Social Model of disability.

- The Medical Model: Traditionally, ADHD or Dyslexia in engineering has been viewed as a "deficit" to be corrected through individual coaching or medication.
- The Social Model: This perspective argues that the developer is "disabled" not by their neurology, but by an IDE designed with a "neurotypical bias". The problem is not the individual's brain, but the environment that fails to accommodate it. This shift in perspective is crucial for designing tools that are not just "assistive" but truly inclusive, as it focuses on modifying the environment (the IDE) rather than trying to fix the individual.

Executive Functioning (EF) Domains in Programming To deepen the analysis of executive dysfunction, we must break it into three core domains:

- Inhibitory Control: The ability to ignore distractions. For ADHD developers, the "pop-up" notifications of a linter can break a fragile state of focus.
- Working Memory: The "mental scratchpad." Dyslexic developers often have a high capacity for 3D spatial reasoning but a limited verbal working memory, making syntax-heavy languages like C++ more taxing than "cleaner" languages like Python.
- Cognitive Flexibility: The ability to switch between thinking about the "big picture" architecture and the "low-level" syntax.

2.3 The "Double-Edged Sword" of Hyperfocus

While neurodivergent developers, particularly those with ASD and ADHD, are often stereotyped as having a "superpower" for deep work, this ability to hyperfocus is more accurately described as a dysregulation of attention. Unlike the psychological concept of "Flow" which is generally sustainable and self-directed, hyperfocus is an involuntary immersion into a task or activity. As noted by [4], hyperfocus allows for an exhaustive exploration of complex problems, yet this intensity often comes at the expense of self-care. Developers may neglect fundamental biological needs, such as nutrition, sleep, and hygiene, leading to rapid burnout.

In the context of software engineering, this creates a High-Intensity/High-Fragility paradox. [10] identifies that for ADHD programmers, “getting in the groove” is a fragile process where a single interruption can collapse hours of built-up mental context. The subsequent state recovery cost is significantly higher for neurodivergent individuals, often leading to burnout or irritability when the state is broken.

In a professional engineering context, this creates a significant conflict between depth and delivery. Hyperfocus can inadvertently trigger over-engineering, where a developer becomes locked into optimising a single component while neglecting broader project milestones or team collaborations. This inability to switch off from an interesting problem makes task-switching, a fundamental requirement of agile development, mentally exhausting and practically difficult. Consequently, what appears to be peak productivity from the outside is often an unsustainable trade-off that risks both the developer’s well-being and the project’s timeline.

2.4 Existing Tools

The design of IDEsoften follows a one-size-fits-all model, which inherently leads to a neurotypical bias, creating barriers for those who may require additional support. [9] observes workplace tools are frequently built with assumptions that "may not align with the needs of neurodiverse employees." IDEs are often packed with tools that enable customisation; however, they are not designed with neurodiverse developers in mind, and complex nested menus can be overwhelming.

Despite the obstacles described, certain features provide essential support for developers. [9] notes that tools such as code completion and linters are vital to alleviate the difficulties associated with dyslexia, such as syntax memorisation and spelling.

However, the burden of making tools accessible often falls to the user. [8] found that developers often create “individual strategies and tool configurations to support their productivity." While these features are currently helping, the reliance on manual customisation implies a lack of "out-of-the-box" inclusivity.

This project aims to bridge the gap between the availability of generic tools and their functional utility. While current tools are excellent at detecting errors, they often fail at explaining them in an accessible manner. By integrating an Error Explainer that decomposes complex compiler messages into manageable tasks, therefore reducing the cognitive load required to debug. This shifts the IDE from a passive environment that requires expert configuration to an active partner in the development process.

2.5 Colour filters

Visual presentation can influence reading performance and comfort.Griffiths et al. [5] found that coloured overlays may reduce visual stress for some readers, although the benefits vary significantly between individuals. This suggests that software tools should support flexible visual customisation rather than relying on a single accessibility configuration.

In some cases, high-contrast models are essential because they provide the necessary visual scaffolding to combat task paralysis. For developers with ADHD, a low-contrast or pastel theme can cause code elements to bleed together, increasing the cognitive effort required to distinguish between keywords, variables, and logic. By utilizing high-contrast ratios—such as 7:1 or higher, per WCAG 2.1 guidelines—the IDE creates a clear visual hierarchy.

This high "signal" environment provides several technical advantages:

- Reduce Visual Search Time: Finding the start and end of a function block becomes instantaneous rather than a manual scan.
- Mitigate the "Blur" of Hyperfocus: As noted by [10], the intense mental strain of being "in the groove" can lead to visual fatigue; high contrast maintains legibility even as physiological focus wanes.
- Support Syntax Logic: In complex, nested code, high-contrast separation between brackets and operators serves as a "visual prosthetic," offloading the work of pattern recognition from the brain to the interface.

The "Halation" Risk and Soft High Contrast While high contrast is vital for clarity, pure high contrast (e.g., stark white text on a pure black background) can be counterproductive for those with ASD or specific visual stresses. This combination can cause halation—a glowing or "bleeding" effect around letters that makes text appear to vibrate or move.

To address this, the most effective models for neurodiversity are often "Soft High Contrast" themes. These utilize high-visibility colors against deep, non-pure-black backgrounds (such as dark greys or navy). This balance provides the necessary "signal" for ADHD developers to maintain focus while preventing the sensory overwhelm and visual distortion common in ASD profiles. However, the implementation must be nuanced. While high contrast is vital for clarity, "pure" high contrast can cause halation (a glowing effect around letters) for those with ASD or visual stress. Therefore, the most effective high-contrast models for neurodiversity are often "Soft High Contrast", utilising high-visibility colours against deep non-pure-black backgrounds.

2.6 AI as a Cognitive Tool: Task Decomposition and Scaffolding

For developers with ADHD and ASD, the most significant barrier to productivity is often not the complexity of the code itself, but the executive function tax required to organise a project. Research by [10] indicates that ADHD professional programmers specifically struggle with "state recovery" and task initiation. LLMs offer a unique solution to this through cognitive scaffolding. This process handles the “low-value” mental energy drains, such as formatting and scheduling, leaving more room for “high-value” deep thinking ([6]).

Chain of Thought (CoT) as Task Decomposition is one of the most effective applications of LLMs for neurodivergent developers. Instead of asking a model to simply “write a function,” the developer can use the model to decompose a broad, abstract goal into a granular, step-by-step plan. This process mitigates the “activation energy” barrier identified in ADHD populations. By prompting a model to “think step-by-step," the AI generates a logical roadmap for the developer to work through, as well as referring back to it if the task is interrupted.

As noted by Mew (2025), this “automated pattern” of scaffolding can increase productivity for developers with ADHD by up to 55 percent. Existing tools already demonstrate this by taking a single overwhelming command (e.g., “build an authentication system”) and breaking it into manageable milestones. This effectively transforms AI from a simple code generator into a “digital ramp” that bypasses the freeze response common in task paralysis ([6]).

2.7 NASA-TLX and SUS: Measuring Cognitive Load and Usability

To evaluate the effectiveness of the plugin, I will use the NASA Task Load Index (NASA-TLX) to measure cognitive load and the System Usability Scale (SUS) to assess usability. The NASA-TLX provides a multidimensional assessment of perceived workload across six dimensions: Mental Demand, Physical Demand, Temporal Demand, Performance, Effort, and Frustration. This will allow me to quantify the reduction in cognitive load achieved by the plugin. The SUS, on the other hand, is a widely used tool for assessing the usability of systems and interfaces. It consists of a 10-item questionnaire that provides a global view of subjective assessments of usability. By using both tools, I can gain insights into how well the plugin reduces cognitive load while also ensuring that it is user-friendly and accessible for neurodivergent developers.

I have chosen these tools because they are well-established in the field of human-computer interaction and provide a comprehensive framework for evaluating both the cognitive and usability aspects of the plugin. The NASA-TLX will help me understand the specific areas where cognitive load is reduced, while the SUS will provide feedback on the overall user experience. This dual approach ensures that the plugin is not only effective in reducing

cognitive load but also meets the usability needs of its target audience.

3 Design and Planning

3.1 User Requirements

The requirements for the plug-in were derived from a synthesis of Cognitive Load Theory and specific neurodivergent accessibility guidelines, notably the "Direct Instruction" and "Visual Comfort" frameworks identified by Rocha [12].

Table 2: Functional and Non-Functional Requirements for the Accessible IDE Plugin

ID	Category	Requirement Description
FR1	Interface	The system shall provide a single-click "Minimalist Toggle" to hide the Activity Bar, Side Bar, Status Bar, and Breadcrumbs simultaneously.
FR2	AI Support	The "Task Architect" shall decompose high-level prompts into a structured checklist of 3–5 manageable sub-tasks.
FR3	Feedback	The "Code Explainer" shall intercept compiler errors and provide non-technical, bulleted summaries (max 3 points).
NFR1	Accessibility	The UI must adhere to WCAG 2.1 Level AA contrast ratios (4.5:1 for text).
NFR2	Cognitive	The system must provide "Non-Destructive Restoration," allowing users to revert all UI changes instantly to prevent anxiety over lost settings.
NFR3	Latency	AI-generated task steps must begin streaming to the UI within 2 seconds of the request to prevent "task-switching" distractions.

3.2 Module Design

The plugin is architected around three core modules, each designed to address specific executive functioning barriers identified in Section 1.3. These modules are unified within the Accessible Dashboard (see Figure [X]), a centralized Webview panel that provides a low-friction entry point to the tool's features.

Minimalist Toggle

The Minimalist Toggle serves as a reset for the IDE. It utilizes a state-management system to track the current visibility of the VS Code workbench. When activated, it dispatches a batch of imperative commands to the vscode.commands API, hiding the Activity Bar, Status Bar, Side Bar, and Breadcrumbs simultaneously. By reducing the total number of visual elements on the screen, the module lowers extraneous cognitive load, allowing the developer to allocate more mental energy to the code editor itself.

Task Architect

The Task Architect is a Human-in-the-Loop (HITL) module that facilitates task initiation

by breaking down overwhelming objectives into granular steps. The module employs a CoT prompting strategy. When a user enters a high-level goal (e.g., "Implement a search bar"), the system instructs the LLM to output a maximum of five actionable sub-tasks. Once the tasks have been generated and optmised for clarity, they are shown in the "Task Breakdown" section of the dashboard. The user is then able to review the steps, there are able to edit or delete any of the steps, this is crucial for neurodivergent developers who may have unique workflows or preferences. Finally, they can click the checkbox to mark the task as complete, graying out the task. By using this process, the AI provides a clear roadmap for task initiation, not providing the code directly. As shown in the "Task Breakdown" section of the dashboard, tasks are rendered as editable fields. This allows the user to modify or refine the AI's suggestions, ensuring the developer maintains agency over their project architecture while the AI handles the "heavy lifting" of the initial decomposition.

Visual Styler

The Visual Styler manages the aesthetic environment of the IDE to reduce eye strain and sensory overwhelm.

WCAG Mapping: The module provides presets (e.g., "High Contrast Dark," "High Contrast Light") that have been mapped to meet WCAG 2.1 Level AA compliance.

Functionality: Beyond simple theme switching, the styler interacts with workspace.getConfiguration to adjust font sizes and line heights. This addresses the "Visual Comfort" framework by ensuring that text-heavy tasks remain legible for developers with dyslexia or light sensitivities.

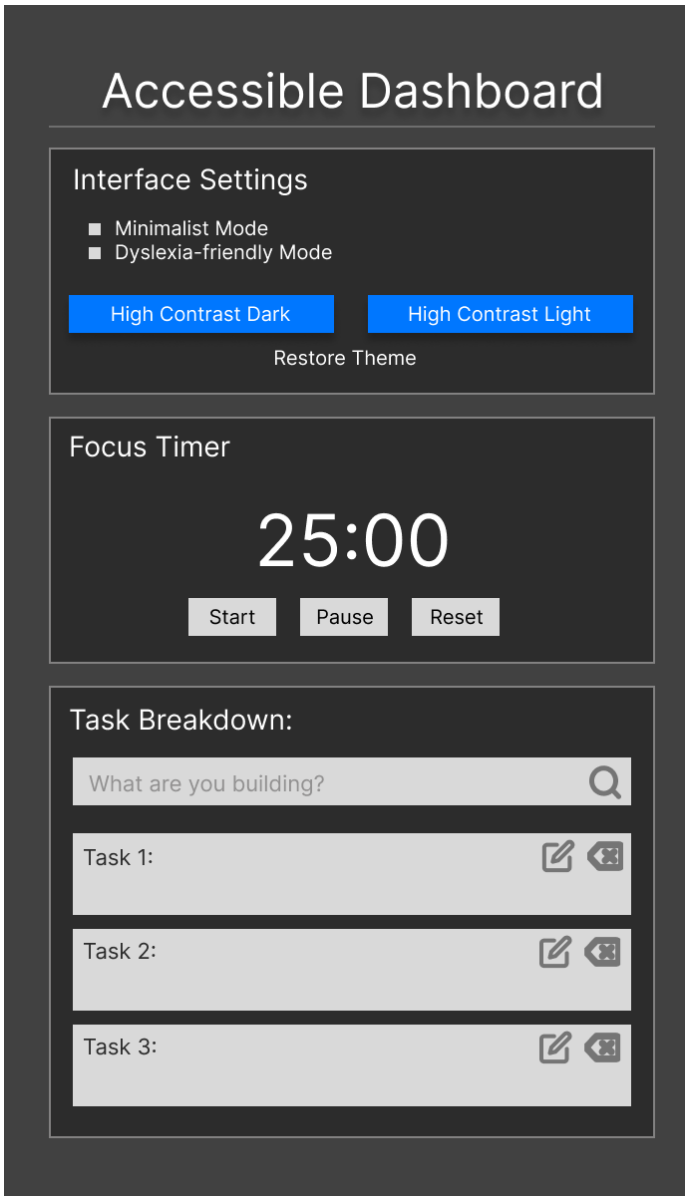

Figure 2: Plug-in UI Plan.

3.3 System Architecture

The system is divided into three primary layers: the Presentation Layer (Webview), the Orchestration Layer (Extension Host), and the External Services Layer (VS Code API).

1. The Presentation Layer: WebView UI

The user interface is implemented as a `WebviewPanel`, which renders a custom HTML/JavaScript dashboard. This allows for a highly flexible UI. This allows for the extension to be customised to best fit the user's needs, predetermined through prior research. This also allows for easier changes as more research is collected in the future

- State Communication: Because WebViews are isolated from the main editor, com-

munication is handled via asynchronous message passing. The UI dispatches JSON-formatted messages using `vscode.postMessage()`, which are then intercepted by the extension's backend.

- Asynchronous Feedback: The UI remains responsive by using event listeners that wait for `updateTime`, `taskResult`, or `errorResult` commands sent back from the extension host.

2. The Orchestration Layer: Extension Host

The core logic resides in the activate function, which serves as the central hub while the extension is active.

- Command Registration: The system registers specific URI commands (e.g., accessible-toggle.showUI) that map user intents to internal JavaScript functions.
- Configuration Management: A key architectural feature is the interaction with vs-code.workspace.getConfiguration(). The extension reads and writes global settings to modify the editor's environment (font size, themes, and layout) in real-time, effectively acting as a wrapper for the VS Code "Settings" engine.
- Context Preservation: To ensure the system is non-destructive, the architecture utilises `context.globalState`. This allows the extension to "remember" the user's original theme and font settings before applying accessibility modifications, enabling a seamless restoration process. This also acts as a form of error prevention in case any settings are applied by accident.

3. The Functional Providers

The architecture employs specialised logic blocks to handle distinct accessibility needs:

Table 3: System Component Interaction and API Dependencies

Component	**Interaction Strategy**	**VS Code API Dependency**
Interface Controller	Imperative Commands	`commands.executeCommand`
Theme/Font Engine	Configuration Updates	`workspace.getConfiguration`
Task Architect	AI Stream Processing	`vscode.lm` (Language Model)
Code Helper	Diagnostic Polling	`languages.getDiagnostics`

4. Integration with Language Models

The "Task Architect" and "Code Helper" features utilise the VS Code Language Model API. This architectural choice ensures security and efficiency, as the extension does not require individual API keys from the user. Instead, it requests access to the editor's

resident models (e.g., GPT-4o). The system uses a CoT prompting strategy to process raw diagnostic data or user goals into structured, neurodiverse-friendly steps before pushing the data back to the Presentation Layer.

3.4 Implementation

3.5 Technology Stack

JavaScript, Node.js, and OpenAI API.

3.6 UI Simplification and Minimalist Toggle

The implementation of the "Minimalist Mode" focuses on reducing visual noise, which is a primary driver of cognitive overload. This is achieved through an imperative command pattern:

- **Workspace Manipulation:** Using `vscode.commands.executeCommand`, the plugin toggles the visibility of the Activity Bar, Side Bar, Status Bar, and Breadcrumbs.
- **Global Configuration:** To disable the Minimap, the system interacts directly with the global user settings via `config.update('editor.minimap.enabled', ...)`.

3.7 AI Integration

The AI-driven features are implemented using a Streaming Data Pattern.

- **Prompt Engineering:** The *Task Architect* uses a strictly defined system prompt to enforce a "Micro-step" structure (3–5 sub-tasks with two 5-minute micro-steps each). This prevents "Wall of Text" responses that can be overwhelming.
- **Code Explainer:** This module polls the `vscode.languages.getDiagnostics` API. It filters for `DiagnosticSeverity.Error`, extracts the error message and line number, and passes this context to the `gpt-4o` model to generate a simplified, non-technical explanation.

3.8 Inital User Testing and Feedback

The Initial user testing was conducted to evaluate the usability and functionality of the plugin. I tested with a developer to get feedback on UI and functionality of the draft

design. At this point the plugin was mostly fully functional, but I wanted to get feedback on the design and usability of the plugin. I asked the user to complete a few tasks using the plugin and then provide feedback on their experience. Some of the tasks that I asked them to complete where changing the theme to a high contrast one, and then changing the font size to 18, this was to test the functionality of the plugin and also to see if it was easy for the user to use. From this, I had three main takeaways:

1. UI and Functionality requirements: The user found that some of the UI elements where not clearly labelled, this caused confusion especially when it came to the "Task Architect"and "minimalist toggle" features. Specifically for the task tool the user was unclear on what to input into the goal fields. When executing the original button label, "Decompose", was too abstract and didn't clearly signal the expected outcome.

To address this, I renamed the action button to "Generate Steps" to create a direct link between the action and the result. I also inproved the placeholder with an example goal (e.g., "Build a login page"). This provides more context for the tool without overcrouding it with a lenghty description.

2. AI Response Integrity There was a technical bug what was truncating the AI response or omitting text contained within quotation marks. This resulted in incomplete micro-steps, significantly reducing the utility of the generated plan. The Solution:

3. Accessibility and Customization Testing To evaluate the plugin‘s versatility and ease of use, the user was tasked with modifying the environment's appearance. The user was first asked to complete the tasks in vanilla VS Code, and then with the plugin enabled. The user was able to successfully switch to a high-contrast theme and adjusted the font size to 18pt in both vedrsions of the IDE. However in in all cases the user found the plugin's interface far quicker to navigate for these specific tasks, as it provided a more direct path to the settings without needing to search through the standard VS Code menus. For example, the user was able to change the theme in aprovimately 1.9 seconds or one click with the plugin, compared to 19.9s or 6 clicks in vanilla VS Code.

Overall the user felt that the plugin would be helpful for developers once the issues with the UI and AI responses were resolved, and that it had the potential to reduce cognitive load and improve task initiation.

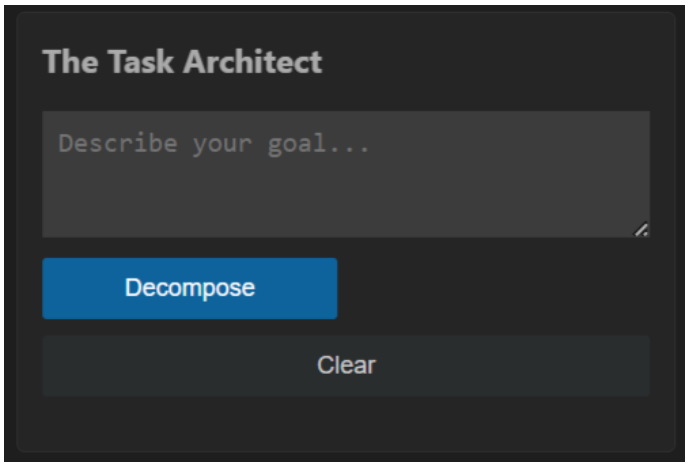

Figure 3: UI at the time of initial user testing, before improvements to the UI and AI response integrity were made.

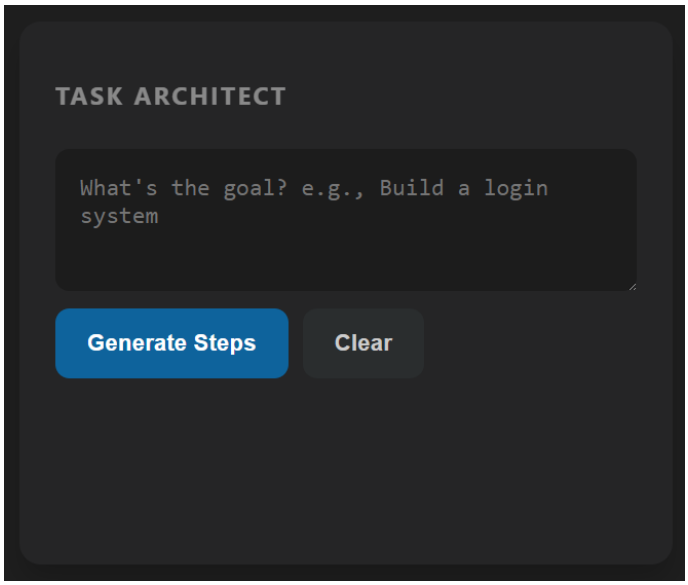

Figure 4: UI after improvements to the UI and AI response integrity were made.

3.9 Evaluation and Methodology

3.10 Experimental Design

This study employs a **within-subjects comparative design**, where each participant acts as their own control. This design is particularly effective for neurodivergent research as it accounts for the high degree of individual variance in cognitive processing styles [16]. Participants will perform two standardized programming tasks: a project-planning phase and a debugging phase.

1. **Control Condition:** Tasks are performed using a standard ("Vanilla") IDE installation.

2. **Experimental Condition:** Tasks are performed using the IDE equipped with the Accessible Plugin features (Minimalist Toggle, Task Architect, and Visual Styler).

To mitigate the **carry-over effect** (where learning from the first task makes the second easier), the order of conditions will be counterbalanced, and task sets of equivalent IL will be used across both conditions.

3.11 Quantitative and Qualitative Frameworks

To evaluate the efficacy of the plugin in reducing EL and supporting executive dysfunction, two validated instruments will be administered.

3.11.1 NASA Task Load Index (NASA-TLX)

The NASA-TLX is a multidimensional assessment tool used to quantify perceived workload. It is uniquely suited for this study as it allows for the isolation of specific cognitive pressures. While all six subscales will be measured to ensure a comprehensive view, the analysis will focus specifically on four key dimensions:

- **Mental Demand:** To assess if the Task Architect successfully reduces the "Executive Function Tax" required for project initiation.
- **Performance:** To measure the user's self-perception of success in the face of syntax errors or logic bugs.
- **Effort:** To quantify the total mental resources required to reach the goal.
- **Frustration:** Critical for neurodivergent populations, this measures the emotional cost of "task-switching" and sensory overwhelm.

Ratings will be collected on a 20-point scale. By comparing NASA-TLX scores across conditions, the study aims to demonstrate a statistically significant reduction in the "Mental Demand" and "Frustration" subscales when using the plugin.

3.11.2 System Usability Scale (SUS)

To assess the reliability and integration of the tool, the SUS will be administered following the Experimental Condition. The SUS is a 10-item questionnaire using a 5-point Likert scale, providing a global view of system usability.

In the context of this study, the SUS serves two primary purposes:

1. **Integration Assessment:** Identifying if the "Minimalist Toggle" feels like a seamless part of the workflow or an added complexity.
2. **Confidence and Learnability:** Measuring the "Activation Energy" required to use the AI features. For a developer experiencing task paralysis, a tool that is perceived as "cumbersome" (Item 8) or "complex" (Item 2) would be counter-productive.

3.12 Data Collection and Analysis

In addition to the questionnaires, the study will record **Time-on-Task** (to measure efficiency) and use a **Think-Aloud Protocol**. This qualitative approach will capture real-time instances of "State Recovery" costs following an interruption, providing insight into how the AI scaffolding supports the developer in regaining hyperfocus.

4 Results and Discussion

4.1 Analysis of NASA-TLX Scores

Did the plugin actually lower "Mental Demand"?

4.2 Task Initiation Findings

Comparison of start times for complex tasks.

4.3 Discussion

How AI-assisted decomposition mitigates hyperfocus/over-engineering.

4.4 Limitations

The subjectivity of self-testing and LLM hallucinations.

References

[1] Linus Bengtsson and Jimmy Hammarberg. *Web Content Accessibility Guidelines (WCAG) within a Company*. 2025.

[2] Jessica Bramham, Fiona Ambery, Susan Young, Robin Morris, Ailsa Russell, Kiriakos Xenitidis, Philip Asherson, and Declan Murphy. "Executive functioning differences between adults with attention deficit hyperactivity disorder and autistic spectrum disorder in initiation, planning and strategy formation". In: *Autism* 13.3 (2009), pp. 245–264.

[3] A. Cao, K. K. Chintamani, A. K. Pandya, and R. D. Ellis. *NASA TLX: Software for assessing subjective mental workload.* [Online; accessed 2026-02-25]. 2009. URL: https://psycnet.apa.org/record/2011-00741-013.

[4] Kiev Gama, Grischa Liebel, Miguel Goulão, Aline Lacerda, and Cristiana Lacerda. *A socio-technical grounded theory on the effect of cognitive dysfunctions in the performance of software developers with ADHD and autism*. IEEE, 2025.

[5] Philip G Griffiths, Robert H Taylor, Lisa M Henderson, and Brendan T Barrett. "The effect of coloured overlays and lenses on reading: a systematic review of the literature". In: *Ophthalmic and Physiological Optics* 36.5 (2016), pp. 519–544.

[6] Connor Harwood. *Navigating the Brain-AI Loop | Remtek Workplace*. [Online; accessed 2026-03-16]. Mar. 2026. URL: https://remtekworkplace.com/knowledge-hub/the-digital-ramp-and-the-cognitive-debt-navigating-the-brain-ai-loop/.

[7] M. Hyzy, R. Bond, M. Mulvenna, L. Bai, A. Dix, and Hunt Leigh S. *JMIR mHealth and uHealth - System Usability Scale Benchmarking for Digital Health Apps: Meta-analysis*. [Online; accessed 2026-02-25]. Aug. 2022. URL: https://mhealth.jmir.org/2022/8/e37290.

[8] Grischa Liebel, Noah Langlois, and Kiev Gama. "Challenges, strengths, and strategies of software engineers with ADHD: A case study". In: *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society*. 2024, pp. 57–68.

[9] Meredith Ringel Morris, Andrew Begel, and Ben Wiedermann. *Understanding the challenges faced by neurodiverse software engineering employees: Towards a more inclusive and productive technical workforce*. 2015.

[10] Kaia Newman, Sarah Snay, Madeline Endres, Manasvi Parikh, and Andrew Begel. ""get me in the groove": A mixed methods study on supporting ADHD professional programmers". In: *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE. 2025, pp. 1217–1229.

[11] Meredith Ringel Morris. “AI and Accessibility: A Discussion of Ethical Considerations”. In: *arXiv e-prints* (2019), arXiv–1908.

[12] Inês Lazana Barros da Rocha. *A Conceptual Model on Neurodivergent Issues in Software Engineering*. 2025.

[13] Barak Rosenshine. “Principles of instruction: Research-based strategies that all teachers should know.” In: *American educator* 36.1 (2012), p. 12.

[14] Judy Singer. “Why can’t you be normal for once in your life? From a ’problem with no name’ to the emergence of a new category of difference”. In: *Disability Discourse*. Ed. by Mairian Corker and Sally French. Buckingham: Open University Press, 1999, pp. 59–67.

[15] John Sweller. *Cognitive load during problem solving: Effects on learning*. 1988.

[16] Pragya Verma, Marcos Vinicius Cruz, and Grischa Liebel. *Differences between neurodivergent and neurotypical software engineers: Analyzing the 2022 stack overflow survey*. Springer, 2025.